Reinforcement Learning from Verifiable Penalties: Process Rewards as Gradient Where Outcome Rewards Are Blind

Bojie Li
Pine AI

Abstract

Outcome-only reinforcement learning—reward the trajectory iff it solves the task—is the dominant recipe for training long-horizon LLM agents, but it has a structural blind spot. Group-relative methods such as GRPO compute an advantage of zero whenever every sampled rollout in a group fails, and on hard tasks the early-training regime is *dominated* by such all-fail groups. The result is a training signal that is silent precisely when learning matters most. We study **Reinforcement Learning from Verifiable Penalties** (RLVP): augmenting the sparse outcome reward with dense, machine-checkable *process* rewards attached at each tool call—a penalty when a move violates a verifiable rule, a credit when it discharges a pending obligation. Because these rewards depend on *how* an episode acts rather than *whether* it ultimately succeeds, they produce gradient from the failed episodes that GRPO discards. We show, on a controlled suite of long-horizon agentic tasks, that this turns a $7\times$ wall of dead updates into smooth learning: RLVP reaches a target success rate in roughly a quarter the episodes of outcome-only GRPO and, unlike GRPO, with near-zero variance across seeds. We isolate *why* through controlled and paired experiments—the credit must be attached to the responsible tokens, not folded into a scalar return—and show the process signal can be *auto-derived* from tool-type metadata and the environment's own error signals, recovering the hand-engineered gain with no human-written rules. We also map the method's boundary honestly. The dynamic-sampling fix of DAPO confronts the same all-fail problem by *discarding and resampling*, which we show pays a $5.6\times$ sampling tax without recovering the lost signal. On a task with a hidden precondition that the policy never spontaneously samples, no reward-only method escapes the wall—a demonstration cold-start does, delineating exactly where self-evolution ends and imitation must begin. And on a real benchmark we trace the method's boundary as a *coverage gradient with an irreducible ceiling*: rules orthogonal to the reward actively *harm*, driving the policy into a degenerate compliance-only optimum; policy-derived rules—procedural *and* verifiable semantic ones—remove that harm; but neither beats outcome-only, because verifiable rules can express policy *validity* while the reward's residual difficulty is task *intent*, which is the reward itself and learnable only from the outcome. Process rewards accelerate the policy-verifiable component of success and cannot substitute for outcome learning of intent.

1 Introduction

The recipe that produced the recent generation of reasoning models is disarmingly simple: sample a group of trajectories, reward the ones that solve the task, and push the policy toward

them with a group-relative advantage. DeepSeek-R1-Zero showed that this outcome-only signal, applied to a base model with no supervised cold-start, is enough to grow sophisticated reasoning behavior [DeepSeek-AI, 2025]. The appeal is that it requires almost no human input—only an automatic checker of final correctness—and it self-evolves from there.

This recipe rests on a quiet assumption: that the base policy already *sometimes* succeeds. Group-relative policy optimization (GRPO) estimates each trajectory's advantage by centering its reward on the group mean [Shao et al., 2024]. When the rewards in a group vary—some succeed, some fail—this is informative. But when *every* sampled rollout fails, the rewards are identical, the mean equals each one, and every advantage is exactly zero. The group produces no gradient. On short-horizon tasks like grade-school math, where a competent base model solves a healthy fraction of problems, all-fail groups are rare and the assumption holds. On long-horizon agentic tasks—a coding agent that must edit, test, and submit across dozens of tool calls, where a single misstep fails the episode—the assumption breaks. The base success rate is low, so most groups are all-fail, and the outcome signal is *silent during exactly the phase of training when the policy most needs to be shaped*.

This paper is about filling that silence with a signal that is cheap, verifiable, and—crucially—present even when the outcome is not. The observation we build on is an asymmetry that practitioners know well but that the reward literature underuses: for an intermediate action, deciding whether it was *right* is hard (did the refund call fail because of the tactic or the representative?), but deciding whether it was *wrong* is often trivial and mechanical (it was the sixth consecutive call to the same number; it deleted a file it never read; it committed without running the tests). Wrongness is verifiable; rightness is not. We attach dense, rule-based rewards at each tool call that exploit this asymmetry—a penalty when a move violates a verifiable rule, and a credit when a move *discharges* a pending obligation (reads a file before overwriting it, runs the tests after a change). We call training with these signals **Reinforcement Learning from Verifiable Penalties** (RLVP).

The reason this helps is mechanistic, not incidental (Figure 1). These process rewards depend on *how* an episode behaves, not on *whether* it eventually succeeds. So in an all-fail group—where GRPO sees only zeros—RLVP still sees that one episode read before writing and another did not, that one ran the tests and another skipped them. That difference is gradient. RLVP extracts a learning signal from the very episodes that outcome-only RL throws away.

Our contributions are a set of findings, each isolated by a controlled experiment, that together characterize when and how this works—and, just as importantly, when it does not.

- **The blind spot is real and the fix is large** (§4). On a calibrated hard task, outcome-only GRPO spends a majority of its early iterations on dead, zero-gradient updates. RLVP reaches a target success rate in roughly a quarter of the episodes, and with near-zero seed variance against GRPO's lottery-like spread.
- **Dynamic sampling does not solve it** (§4). DAPO's strategy of discarding all-fail groups and resampling [Yu et al., 2025] removes dead updates but pays a 5.6× sampling tax for the discarded rollouts—worse, on the fair axis of episodes *generated*, than vanilla GRPO. It relocates the cost rather than recovering the signal.
- **Credit placement is load-bearing, and the signal can be free** (§5). The gain requires attaching process credit to the *responsible tokens*; folding it into a scalar return is twice as slow. And the entire process signal can be *auto-derived* from tool-type tags and the

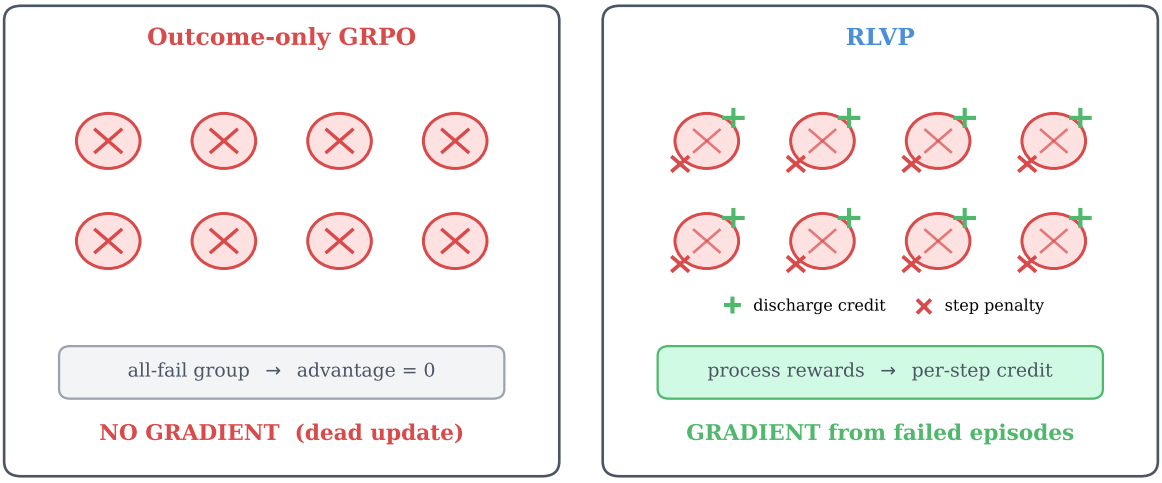

Figure 1. Why process rewards help where outcome rewards cannot. On a hard task, most early-training groups are *all-fail*: every rollout in the group fails the task. *Left:* for outcome-only GRPO, identical (zero) rewards yield a group-mean-centered advantage of zero on every token—no gradient, a *dead update*. *Right:* RLVP attaches verifiable penalties (×) and obligation-discharge credits (+) to individual tool calls. The failed episodes now differ in their process signal, so the update is non-zero. RLVP turns the discarded all-fail batch into a learning signal.

environment's own error codes, recovering the hand-engineered efficiency with no human-written rules—a maximally self-evolving variant in the spirit of R1-Zero.

- **Self-evolution has a discovery ceiling** (§6). On a task whose solution hinges on a precondition the policy never spontaneously samples, no reward-only method—outcome, DAPO, or RLVP—ever gets off the ground. A single synthesized demonstration breaks the wall, cleanly marking the boundary between what reward can grow and what imitation must seed.
- **The boundary is a coverage gradient with an irreducible ceiling** (§7). On a real benchmark, generic rules orthogonal to the reward actively harm (RLVP collapses into a degenerate policy); policy-derived rules—procedural *and* verifiable semantic ones—remove the harm; but neither beats outcome-only, because verifiable rules can cover policy *validity* yet not task *intent*, which is the reward itself and learnable only from the outcome.

2 The Blind Spot of Outcome-Only RL

Setup. We study tool-using agents in deterministic, fully verifiable environments so that every claim about gradient and reward is exact rather than estimated. Our primary testbed is a family of *N-stage chained tasks*: an agent must complete N independent file-operations subtasks (edit a config value, clean up temporary files, create a file) in a single episode, using tools `list_dir`, `read_file`, `write_file`, `delete`, `run_tests`, `submit`. Success requires all N stages correct. Because per-stage success compounds multiplicatively, N tunes base difficulty smoothly: a four-stage task (N=4) sits at roughly 8% base success, a six-stage task near 2%. All experiments train Qwen3-4B [Team, 2025] with our own GRPO implementation; efficiency is always measured in *episodes generated* so that resampling costs are counted, not hidden.

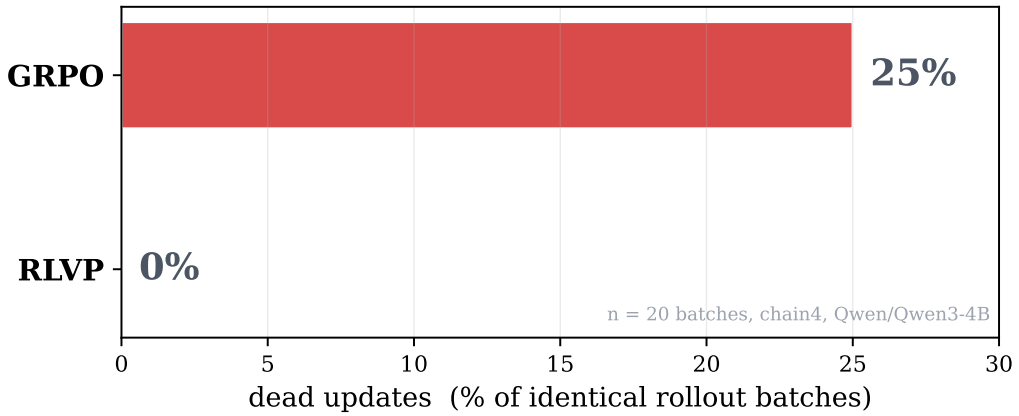

Figure 2. The mechanism, measured in a paired experiment. We generate a batch of rollouts *once* and score it under two credit schemes. Outcome-only credit yields a dead, zero-gradient update on one in four identical batches; RLVP's credit is never dead, because its process channel produces gradient from the failed episodes that the outcome channel reduces to zero. Same rollouts, opposite outcomes: the effect is causal, not an artifact of different sampling.

Dead updates are not a rounding error. On N=4, outcome-only GRPO begins training with the large majority of its groups all-fail. We instrument the trainer to log, per iteration, whether the update was *dead*—whether the entire batch yielded zero advantage and thus no optimizer step. Across a 60-iteration run, outcome-only GRPO logs roughly two-thirds of its iterations as dead; on a single batch where all eight rollouts of all eight tasks fail, the update time drops to literally zero. The policy generated 512 rollouts that iteration and learned nothing from them.

A skeptic might object that this is a property of our particular rollouts, not the credit scheme. We rule this out with a *paired* experiment (Figure 2): we sample a batch of rollouts once and then apply both the outcome-only and the RLVP credit assignment to the *identical* batch. Outcome-only credit produces a dead update on one in four batches; RLVP's credit is never dead. The difference is the credit scheme, holding the data fixed. This is the cleanest statement of the thesis: on the same failed episodes, outcome reward sees nothing and process reward sees structure.

3 Reinforcement Learning from Verifiable Penalties

Rules as a richer verifier. R1-Zero needs one piece of machinery: an automatic checker of final correctness. RLVP needs a slightly richer one—a checker that can also fire mid-trajectory. A *rule* is a deterministic predicate over the pre-call environment state and the proposed tool call, returning a fixed penalty when violated. A paired *discharge* predicate fires when a call satisfies a previously-pending obligation. For our file environments the rules are: do not overwrite a file you never read; do not delete a path you never inspected; do not submit after a change without running the tests; do not reissue a call that has already errored twice. Each has a natural discharge: reading a file, listing a directory, running the tests after a mutation. Crucially these are *specification*, not *demonstration*—written once, requiring no labeled trajectories and no ability to solve the task. They are the analog of R1-Zero's verifier, not of a supervised cold-start.

Two-channel credit. RLVP carries two reward channels with separate normalizers. The *outcome channel* is the sparse terminal reward, group-mean-centered as in GRPO and applied to every action token of an episode. The *process channel* attaches, to the tokens of each individual tool call, a penalty $-\lambda$ per violation and a credit $+\beta$ per discharged obligation. The two channels are normalized by their own token counts before being summed into a per-token advantage and optimized with a standard clipped policy-gradient objective. The separation matters: a single global normalizer would divide the sparse process signal by the full batch token count, and we found this makes the penalty gradient collapse by two to three orders of magnitude exactly when the outcome channel saturates—vanishing when it is needed most. Per-channel normalization is the operational content of "token-attached" credit, and §5 shows it is load-bearing.

Annealing. The process channel is a means, not an end: we want the policy to internalize the discipline, not to be permanently coerced by it. We anneal $\lambda, \beta \to 0$ once compliance saturates and continue on the outcome channel alone. Persistent compliance after annealing is evidence the behavior is in the weights; §5 shows annealing is also what protects the converged *outcome* from a degenerate compliance-only optimum.

4 Process Rewards Make Outcome Learning Sample-Efficient

The central empirical claim is about the outcome itself: with the same episode budget, RLVP reaches a given task-success level in far fewer samples than outcome-only RL. Figure 3 shows held-out success against episodes generated for five training methods on N=4. Outcome-only GRPO crawls—it spends most of its budget on dead updates and only escapes near the end of the run. RLVP rises immediately, because its process channel converts the early all-fail groups into gradient. The two process-reward baselines we reimplement, GiGPO-style step advantages [Feng et al., 2025] and StepTool-style per-call rewards [Yu et al., 2024], also beat outcome-only by a wide margin—the broad lesson is that *any* dense verifiable signal helps—but RLVP is fastest to threshold.

The gain is large and consistent; DAPO's is neither. Figure 4 aggregates the headline comparison over three seeds. RLVP reaches 50% success in a number of episodes that is not only far smaller than GRPO's but has essentially *zero* variance across seeds, whereas outcome-only GRPO is a lottery—sometimes lucky and fast, sometimes stuck for thousands of episodes, depending on when it first stumbles into a non-all-fail group. This consistency is a practical property in its own right: outcome-only RL on hard tasks is not just slow on average, it is unpredictable.

DAPO [Yu et al., 2025] was introduced to address precisely the all-fail-group problem, via *dynamic sampling*: discard any group whose accuracy is 0 or 1 and resample until the batch is full of informative groups. This guarantees every gradient step is non-dead—but it does so by throwing away the failed rollouts and paying to regenerate. The discarded rollouts are sunk cost, and on hard tasks the oversampling factor explodes. Figure 4 shows DAPO paying a 5.6× sampling tax and, on the fair axis of episodes generated, ending up *slower than vanilla GRPO*. The contrast with RLVP is the conceptual heart of the paper: confronted with an all-fail group, DAPO says "this group carries no information, discard it," while RLVP says "this group carries

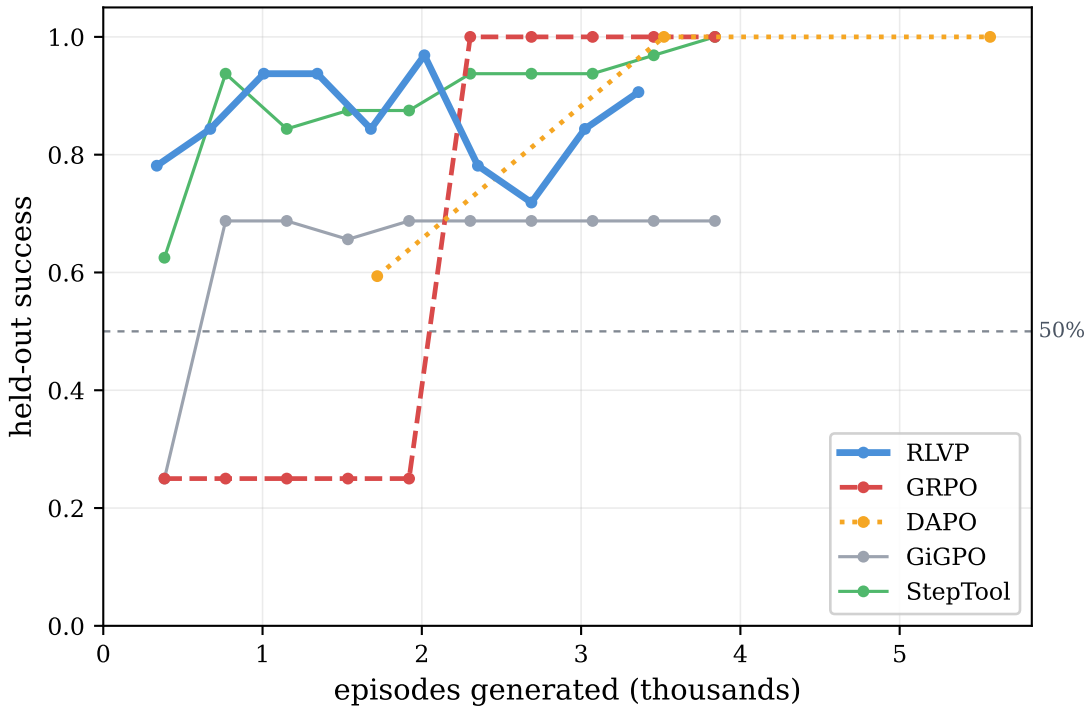

Figure 3. Sample efficiency on the calibrated hard task (N=4), held-out success vs. episodes generated. Outcome-only GRPO sits near the base rate for most of training—it is starved of gradient by all-fail groups—then converges only at the end. RLVP climbs immediately by learning from those same failed groups. Dense-process baselines (GiGPO, StepTool reimplementations) also far outpace outcome-only; RLVP is fastest to the 50% threshold. The x-axis counts episodes *generated*, the fair currency that includes DAPO's resampling.

no *outcome* information but plenty of *process* information." On a binary reward all failures are identical, which is exactly why DAPO can only discard them; verifiable process rewards break that degeneracy and turn the failures into signal.

5 An Anatomy of the Gain

A method with several components invites the question of which one does the work. We answer it by ablation on N=4 (Figure 5), and the answers revise some natural intuitions.

The token-attached channel is the efficiency lever. Replacing token-attached process credit with the same rule rewards *folded into the scalar return* (a "naive" dense-credit baseline) doubles the episodes to threshold. Placing the credit on the responsible tokens—not merely *having* the dense reward—is what buys the speed. This is the empirical reason per-channel normalization, not a global one, is non-negotiable.

Annealing protects the converged outcome. Removing annealing does not slow learning; it lowers the *ceiling*. Without it, the process pressure never relaxes and the policy drifts into over-compliant, bloated episodes whose final success is markedly worse. Annealing is thus doing double duty—an internalization probe and a guard against a compliance-only optimum,

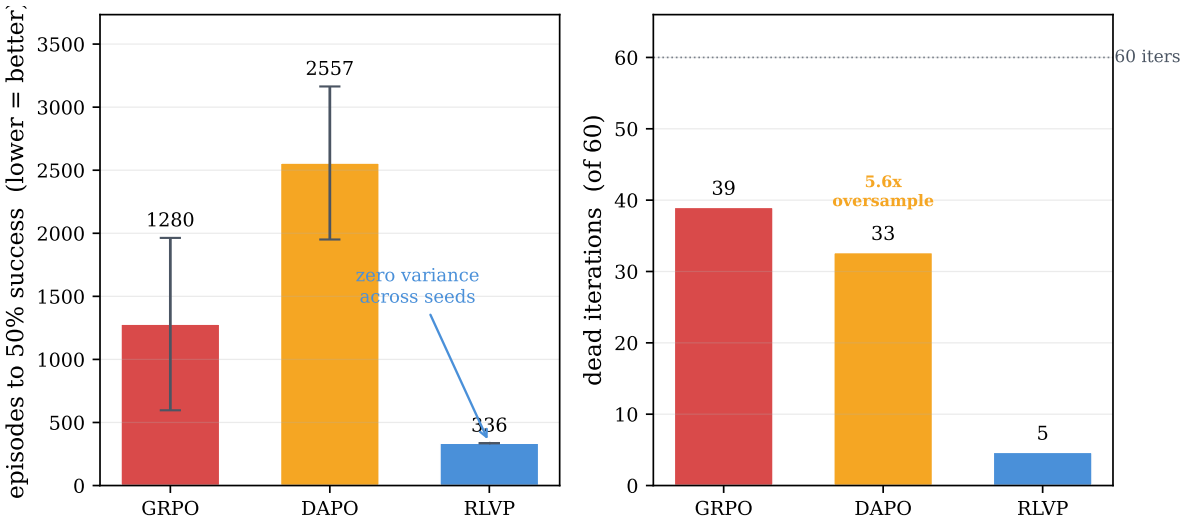

Figure 4. Headline comparison over three seeds. *Left:* episodes to reach 50% success (lower is better). RLVP is several times faster than GRPO and DAPO and—unlike both—has near-zero seed variance; GRPO's enormous error bar reflects the all-fail-group lottery. *Right:* the mechanism behind the gap. GRPO and DAPO burn most iterations on dead or stalled updates, while DAPO additionally generates 5.6× more episodes than it uses. RLVP wastes almost no updates and no extra samples.

a failure mode that returns with force in §7.

The discharge credit is the positive signal that escapes all-fail groups. Penalties alone suppress wrong moves but cannot *induce* a never-tried right one; the obligation-discharge credit is the directional, positive process signal. Removing it leaves more dead iterations and a lower ceiling. We also find a clean fairness result: a control that gives the outcome-only policy the *same synthesized demonstrations* as a mixing-based variant, but no process channel, helps—demonstrations are real signal—yet the process channel alone, with no demonstrations, is faster still. The gain is not "demonstrations in disguise."

The process signal can be free. The strongest form of the R1-Zero ideal is to need no hand-written rules at all. We show this is largely attainable. We replace every hand-written predicate with three *universal* meta-rules derived only from per-tool *category tags* (observe / mutate / verify / terminal) and the environment's own error signals: do not mutate a target you have not observed; do not terminate without verifying since the last mutation; do not repeat a call the environment has errored on. The category tags are task-agnostic metadata that real tool schemas (OpenAPI, function-calling specs) frequently already expose. This auto-derived process reward recovers the hand-engineered efficiency almost exactly—matching hand-written rules and remaining roughly 7× faster than outcome-only—with no task-specific human input. RLVP's "specification" cost can, in well-structured environments, be paid by metadata that already exists.

6 The Ceiling, and the Limit of Self-Evolution

Sample efficiency is a statement about *speed* to a ceiling that, on the chained tasks, every method eventually reaches: chained tasks are compositionally easy—master the per-stage

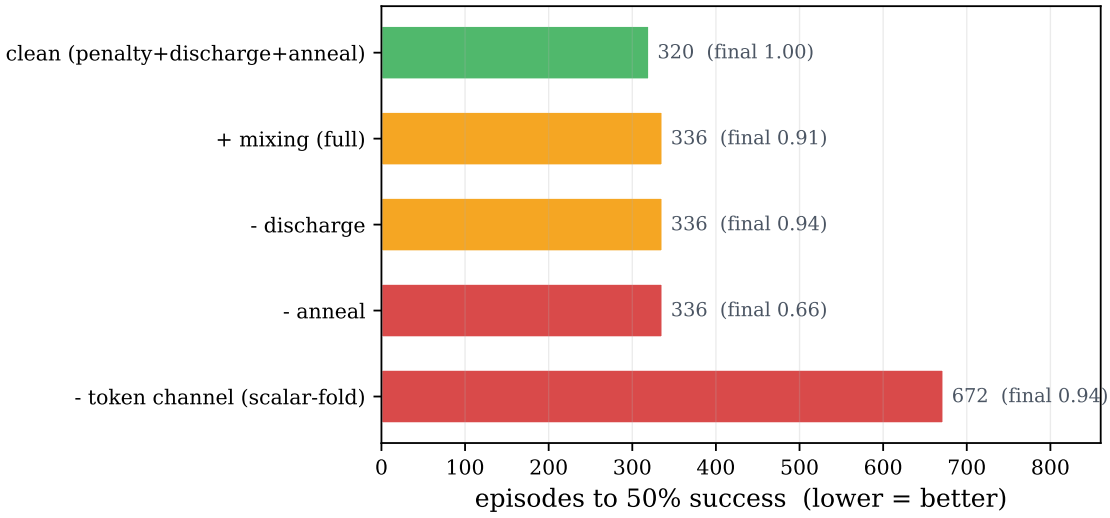

Figure 5. Component attribution on $N{=}4$ (episodes to 50% success; final converged success annotated at each bar). The token-attached credit channel is the efficiency lever—folding the same rewards into a scalar return doubles the cost. Annealing protects the ceiling (its removal halves final success). The clean recipe (penalties + discharge + token channel + anneal) is the best variant; a per-step length cost, helpful only on very long horizons, hurts here.

pattern and all stages follow—so given enough budget even outcome-only GRPO converges. To ask whether process rewards can raise the *ceiling*, not just the speed, we need a task that the model cannot saturate at all.

We construct one with a property that distinguishes *hard-to-discover* from *hard-to-compute*. RLVP's rules guide *procedure*, not answers, so a task that is hard because the answer is hard would not test it. Instead we build a *silent precondition gate*: to write a protected file the agent must first read an access-control file and request access; skip either step and the write silently no-ops—it reports success but does not take effect, and the task fails with no hint why. Calibration confirms the trap is total: the base model solves the task zero percent of the time, and never reads the access-control file—it discovers the obvious `request_access` tool but never the hidden precondition, even when the rule is spelled out in its prompt.

The result (Figure 6) is sharper than "RLVP wins." On this task *every* reward-only method fails to zero—outcome-only, DAPO, and clean RLVP alike. The reason is precise and is the same all-fail mechanism viewed from a new angle: RLVP's discharge credit for reading the access file can only reward that behavior if the policy *samples* it, and the policy never does. A penalty can suppress a wrong move, but no purely on-policy reward can induce a behavior that is never tried. This is the discovery ceiling of self-evolution.

What breaks the wall is a single *demonstration*: we inject one rule-engine-synthesized compliant trajectory into each group. Training on it—through the process channel only, so an imperfect demonstration would still teach the workflow without teaching a wrong answer—drives the policy across a sharp phase transition to perfect, generalizing success, where prompting the very same behavior had failed. The demonstration changes the weights; the prompt only asks. This delineates the boundary cleanly, and it maps onto the R1-Zero/R1 distinction: reward-only self-evolution suffices when the required workflow is *discoverable* (the chained tasks, where clean RLVP is best and demonstrations are redundant), but a demonstration cold-start becomes *necessary* when a precondition is *un-discoverable*. Mixing is not a free

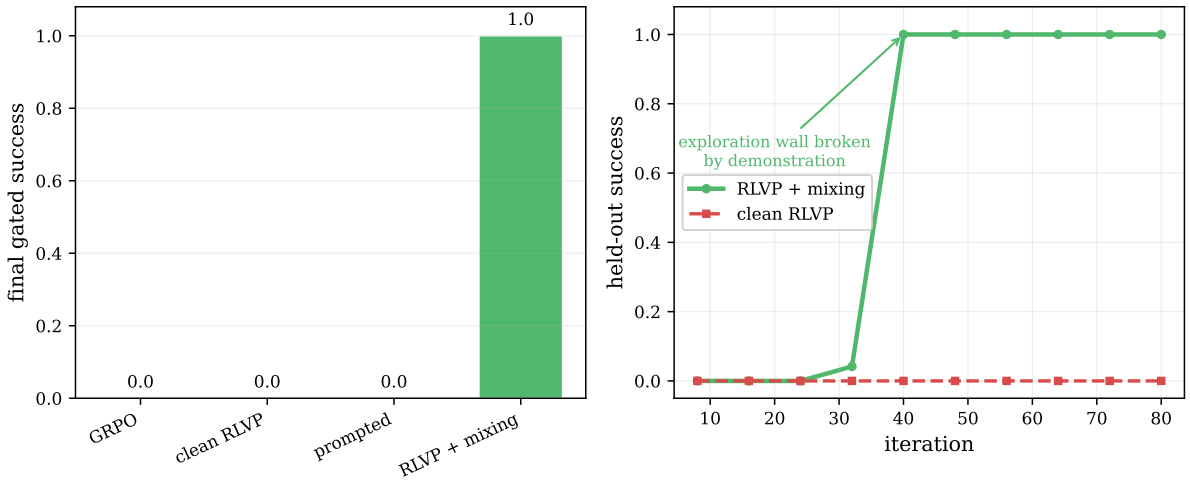

Figure 6. A non-saturating ceiling test, and the discovery wall. *Left:* on the silent-gate task, every reward-only method—outcome, DAPO, clean RLVP, even a rules-in-prompt policy—converges to zero, because the required precondition is never sampled and so the discharge credit can never fire. Adding a single synthesized demonstration breaks the wall and the policy reaches perfect success. *Right:* the demonstration-mixing run shows a sharp phase transition—flat at zero, then a jump to perfect held-out success—while clean RLVP stays at the floor.

improvement to be applied everywhere; it is the specific tool for the specific regime where exploration cannot reach.

7 When Process Rewards Help the Outcome: The Alignment Boundary

Every result so far lives on tasks where the verifiable rules *are* the bottleneck to success—read before write, run the tests, pass the gate. There the process channel and the outcome point the same direction, and accelerating compliance accelerates success. The honest question is what happens when they do not, and to answer it we move off our synthetic suite onto τ-bench-style airline customer service [Yao et al., 2024], a real benchmark whose reward depends on *semantic* policy adherence—authorization, refund eligibility, confirmation—rather than on the structural ordering our generic rules encode.

We ran this in three tiers, and the result (Figure 7) is more informative than a clean negative. In the first tier, RLVP with *generic* structural rules—the same read-before-write hygiene that works on the chained tasks—does the opposite of learning: it drives its (now orthogonal) violations to zero and its task reward collapses to the floor. The policy discovers that the cheapest way to never violate a rule about, say, confirming before calling is to not place the risky calls at all; it degenerates into a compliant, idle, useless policy. This is the *compliance-only attractor*, the very failure mode annealing exists to prevent—but here it arrives within the first handful of iterations, too fast for a typical anneal schedule to catch, and once the policy is at zero reward there is no success for the outcome channel to recover from. Misaligned rules do not merely fail to help; they *harm*.

The cause is *rule–outcome misalignment*, and we test that diagnosis directly. We compile a second rule set from the benchmark's own policy document—obtain the user id and look up the reservation before modifying it, confirm before writing—with discharge credits for those prerequisite lookups. These rules are outcome-instrumental by construction: a correct

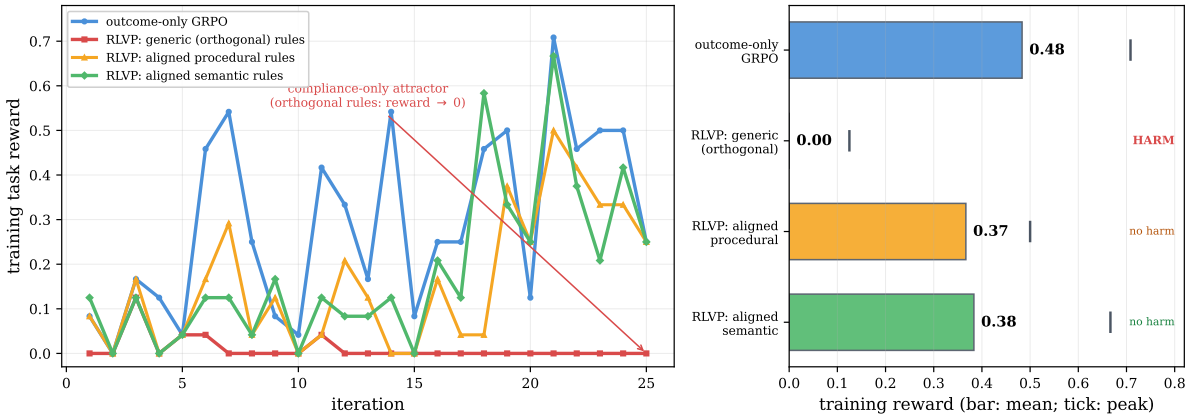

Figure 7. The coverage gradient, and where it saturates, on τ-bench airline. *Left:* training task reward for four signals. Outcome-only GRPO (blue) learns the benchmark. RLVP with *generic* rules orthogonal to the reward (red) collapses into the compliance-only attractor. RLVP with policy-derived *procedural* rules (orange) and with verifiable *semantic* content-validity rules (green), both with early annealing, do not collapse. *Right:* mean training reward per tier (bars) with peak (ticks). Misaligned rules *harm*; aligned procedural and aligned semantic rules *remove the harm* and the semantic peak nearly reaches outcome-only's, but neither *beats* it on average. The gradient saturates: verifiable rules cover policy *validity*, but the residual gap is task *intent*—which permissible change *this* user wants—and that is the reward itself, not a general rule.

modification *requires* the looked-up state, so the discharge credit pulls toward the productive workflow rather than rewarding idleness, which is exactly what should close the attractor. Paired with an earlier anneal, it does (the green trajectory in Figure 7): the collapse is gone, the reward holds, and violations still reach zero. Alignment plus early annealing converts *harm* into *no harm*.

It does not, however, convert it into *gain*: aligned RLVP matches but does not beat outcome-only. The natural next hypothesis is that our policy-derived rules are still merely *procedural*—they govern the order of operations—while the benchmark's reward turns on *semantic* correctness, and that rules covering the semantics would close the gap. We test it with a third rule set: verifiable *content-validity* checks against the live database—do not modify a basic-economy reservation, do not change the passenger count, do not use a payment method absent from the user's profile. Each is a policy constraint the API does not enforce, so a modification violating it is guaranteed to fail; penalizing it prunes failure-bound actions. These rules do fire in training (the agent attempts such invalid modifications early, then learns to avoid them), and they help the *ceiling*—the best iterations climb to nearly outcome-only's peak. But on average they still do not beat outcome-only (Figure 7, right).

The reason is the most important conclusion of this section: the coverage gradient *saturates*. Our semantic rules cover policy *validity*—whether a modification is permissible—but the residual difficulty is task *intent*: which *specific* permissible change *this* user wants. That is not a general verifiable rule; a rule encoding it would simply be the task specification, i.e. the reward itself. Verifiable process rewards can thus express the part of success that is *general policy*—procedure and validity—but not the part that is *task-specific intent*, and on this benchmark the latter dominates and is learnable only from the outcome signal. The boundary is therefore not "find ever-better rules" but an *irreducible* one. RLVP accelerates the policy-verifiable component of success and cannot substitute for outcome learning of intent—which is precisely why it

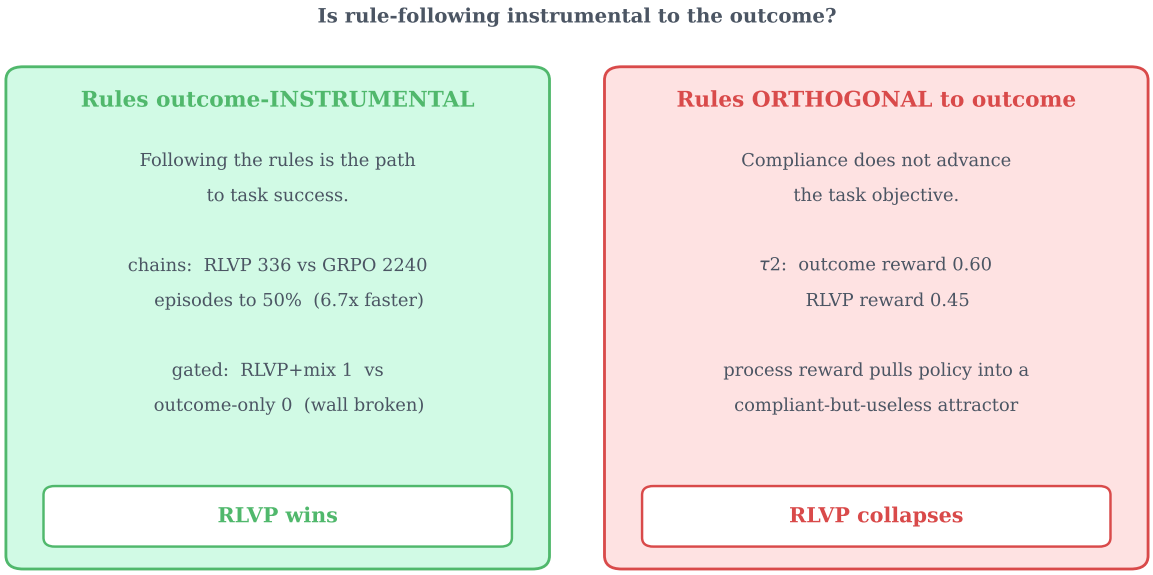

Figure 8. The boundary of the method, as a single axis: are the rules outcome-instrumental? When the verifiable rules are aligned with what success requires (chained tasks: the workflow is the task; gated task: the rule *is* the hidden precondition), RLVP converts process compliance into large outcome gains. When the rules are orthogonal to the reward (τ-bench with generic structural rules), the same machinery optimizes compliance into a degenerate policy. Process rewards accelerate the outcome only on the left.

shines on the chained and gated tasks, where the verifiable workflow *is* essentially the whole task, and only neutralizes harm on a benchmark where it is not.

8 Verifiable Rules vs. a Learned Self-Critic

The boundary above shows what verifiable rules *cannot* reach—task intent. This invites the obvious alternative: rather than hand-specify rules, let the policy *judge itself*, in the lineage of self-rewarding language models and RL from AI feedback [Yuan et al., 2024, Lee et al., 2023, Bai et al., 2022, Zheng et al., 2023]. If a model can flag its own bad tool calls in natural language, perhaps a learned self-critique can supply the dense process signal with no rules at all—and even reach the intent that rules cannot. We test this directly, under one constraint that keeps the comparison honest: the critic is the *same model* as the policy (same weights, no larger judge), so any signal is genuine on-policy self-reflection rather than distillation. The question is whether *verifiability* is essential to a process reward or merely convenient. The answer sharpens the admissibility view of §3: a learned proxy does not remove reward-hacking, it relocates it one level up. Figure 9 maps the outcomes across two axes—whether a verifiable rule is cheaply specifiable, and whether the on-policy critic can detect the violation from the trajectory—and the rest of this section fills in each quadrant.

As a *detector*, self-critique has a structural blind spot. We first ask, offline, whether the model can recover its own rule violations when shown only the domain tools and its own transcript—no rules. Blind self-critique reliably flags violations that are *self-evident from the trajectory surface* (calling before doing any lookup), but is essentially blind to *stateful-bookkeeping*

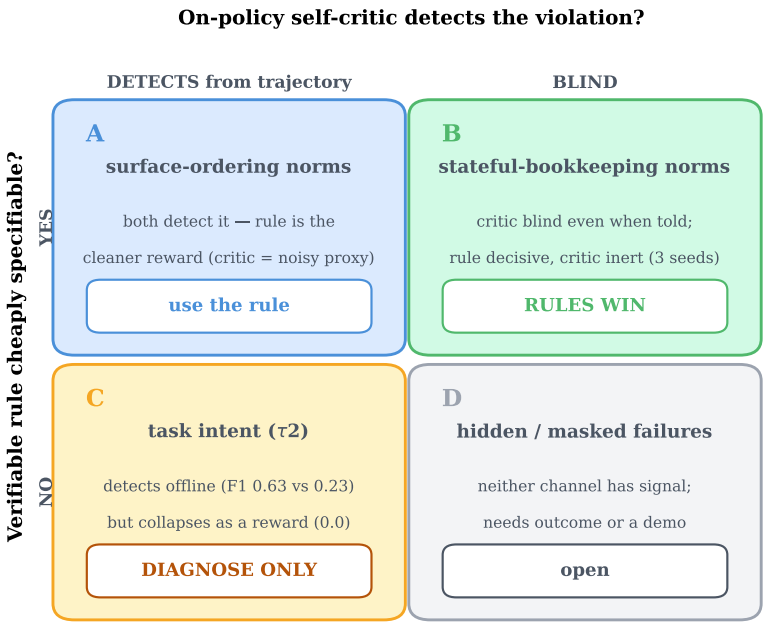

Figure 9. When a verifiable rule beats a same-model self-critic, and when neither trains. The two axes are whether a verifiable rule is cheaply specifiable and whether the on-policy self-critic can detect the violation from the trajectory. **(A)** surface-evident norms: both channels see the violation, but the deterministic rule is the cleaner reward. **(B)** stateful-bookkeeping norms: the critic is blind even when told the rule, and as a dense reward a rule with *identical* credit structure and *perfect* recall beats both the live and a frozen self-critic on every seed—isolating the cause as the critic's imprecision, not its non-stationarity. **(C)** task intent (τ-bench): no cheap rule reaches it; the self-critic is a useful *offline* failure detector (F1 0.63 vs. rules 0.23) but *collapses* when used as a training reward. **(D)** failures masked by the environment: neither channel has signal. The learned critic is a poor training reward in every quadrant; its only robust use is offline diagnosis (C).

norms that require reconstructing history—did it read *this* file before overwriting it, did it test after its *last* mutation (recall ≈ 0)—and remains so *even when handed the rules*. Judging a fixed set of trajectories with critics of increasing size (1.7B, 4B, 8B) shows the blind spot is structural, not a capability gap: per-rule recall on the stateful norms stays near zero across all three, and overall recall is flat ($0.46, 0.38, 0.41$). On-policy self-critique does not scale. (The one violation whose recall *does* rise with critic size is one whose consequence is masked in the environment; closing it requires a larger judge—i.e. distillation, which the constraint forbids.)

As a *reward*, a rule beats the self-critic even at equal structure and perfect recall. We replace the rule channel with the same model's blind self-critique (the per-turn penalty now fires on the turns the critic flags) and train, in the all-fail regime where the process channel is the only gradient (Table 1). A penalty-only *rule* with the identical credit structure is decisive on every seed—it drives violations to near zero when the policy stays stuck, or pushes it out of the all-fail regime entirely to 99% success. The *self-critic*, with the same structure and *perfect recall* against the rule oracle, is inert on both axes on every seed. Crucially, a *frozen* critic (a never-updated copy of the base model, hence a stationary reward) fails identically to the live one: the gap is not the critic's co-evolution with the policy but its imperfect *precision*—a $\sim$6%

Table 1. A deterministic rule beats a same-model self-critic as a dense reward, in the all-fail regime (consumer-service domain, 1.7B, three seeds; late-training means ± std). The rule and both critics share the identical penalty-only credit structure; the critic additionally has *perfect recall* against the rule oracle. The rule is decisive every seed (violations→0 when stuck, or escape to high success); the critics are inert. The *frozen* (stationary) critic fails like the live one, isolating the cause as the critic's ∼6% false-positive imprecision rather than non-stationarity.

process reward	detection (P/R)	violations/episode	task success
rule penalty (deterministic)	1.00/1.00	$\mathbf{0.38 \pm 0.44}$	$\mathbf{0.33 \pm 0.47}$
self-critic, live (co-evolving)	0.94/1.00	0.94 ± 0.05	0.00
self-critic, frozen (stationary)	0.94/1.00	0.93 ± 0.02	0.00

Table 2. Self-critique is a usable intent *detector* but a failed intent *reward* (τ-bench airline, Qwen3-4B). *Left:* as a failure predictor on rule-clean (intent) failures the self-critic far exceeds the verifiable semantic rules (3 rollout seeds). *Right:* as the dense training reward, the same self-critique collapses, while outcome-only and the rules both learn (training reward, early→late; 20 iterations). The intent signal is real for diagnosis but unusable as a gradient.

offline: failure prediction		*trained as reward*	
channel	F1	channel	reward (early→late)
semantic rules	0.23 ± 0.06	outcome-only	$0.09 \rightarrow \mathbf{0.50}$
blind self-critique	$\mathbf{0.63 \pm 0.02}$	semantic rules	$0.10 \rightarrow 0.35$
		blind self-critique	$0.01 \rightarrow \mathbf{0.00}$

false-positive rate that penalizes correct actions, with nothing to wash it out when the process channel is the sole gradient. The gap is also regime-specific: repeating the comparison at 4B, where the tasks become solvable and the outcome channel dominates, the distinction washes out—consistent with the thesis that process rewards matter most precisely on the all-fail groups outcome-only RL cannot use.

At the intent frontier, self-critique *detects* but cannot *train*. Finally we return to τ-bench, where rules cannot reach intent. Here the self-critic does have signal the rules lack: offline, on episodes that failed but are rule-clean—where the verifiable rules are silent by construction—blind self-critique flags a mistake 42% of the time, and as a predictor of episode failure it reaches $\text{F1} = 0.63 \pm 0.02$ against the semantic rules' 0.23 ± 0.06 (Table 2, left). But that diagnostic signal does not convert into a training reward: used as the dense channel it *collapses* the policy to zero reward, far below outcome-only and below even the verifiable rules (Table 2, right). The noisy per-turn critic penalties corrupt the graded outcome signal rather than complementing it.

Synthesis. Across the map, a learned same-model self-critic is a poor *training* reward: where a verifiable rule exists it loses to one even at equal structure and perfect recall (its imprecision is fatal once the process channel is the only gradient), and where no rule exists it can flag intent failures offline but collapses when used as a gradient. Its only robust value is offline diagnosis, and it neither scales with critic size nor bootstraps as the policy improves (the critic's agreement with the oracle stays flat through training). This is the un-gameability criterion of §3 applied to a *learned* signal: defining a progress reward by a learned proxy relocates the

credit-assignment problem into the proxy, which is then either too imprecise to train against or—at the intent frontier—reduces to approximating the value function itself, the same wall as "intent is the reward, learnable only from the outcome." Verifiability is not a convenience; it is what makes the dense channel safe to optimize. (These results are centered on a 1.7B policy with a 4B replication, in the regime where the outcome channel is blind; the τ-bench training comparison is short and single-seed, so we read its *collapse* as the result, not its magnitude.)

9 Related Work

Outcome RL for LLMs and its credit-assignment problem. GRPO [Shao et al., 2024] and the R1 line [DeepSeek-AI, 2025] established outcome-only RL with verifiable rewards as the dominant agent-training recipe. The credit-assignment difficulty on long horizons is now a recognized subfield, with step-level value estimation and turn-level reward shaping among the proposed remedies [Feng et al., 2025, Yu et al., 2024]. We differ in framing the problem as the *all-fail-group blindness* of group-relative methods and in showing that a *verifiable, non-learned* process signal—rather than a learned process reward model—fills exactly that gap. DAPO's dynamic sampling [Yu et al., 2025] is the closest direct comparison; we show it relocates the all-fail cost into sampling rather than recovering the discarded signal, and that the two are complementary in principle (DAPO's clip-higher and token-level loss are orthogonal to our credit scheme).

Process rewards, rubrics, and tool-use RL. Step-grained tool-use RL [Yu et al., 2024] and fine-grained tool reward design [Qian et al., 2025] attach rewards at the call boundary, as we do; our emphasis is the asymmetry between verifiable wrongness and unverifiable rightness, the discharge-credit construction, and the auto-derivation of the signal from tool metadata. Rubric- and verifier-based rewards [Gunjal et al., 2025] extend verifiable RL beyond automatically-checkable domains; our rules are a deterministic, online instance applied per tool call.

Learned rewards and self-critique. Self-rewarding language models [Yuan et al., 2024], RL from AI feedback [Lee et al., 2023], Constitutional AI [Bai et al., 2022], and LLM-as-judge evaluation [Zheng et al., 2023] replace human or rule-based reward with a model's own judgments. Our ablation (§8) is a controlled, same-model instance of this idea used as a *process* reward, and our finding is a cautionary one: a learned self-critic is dominated by a deterministic verifiable rule even at equal credit structure and perfect recall, and collapses where no rule applies—defining process reward by a learned proxy relocates reward-hacking rather than removing it. This motivates our emphasis on *verifiable*, non-learned process signals.

Demonstrations, self-evolution, and the discovery wall. R1-Zero [DeepSeek-AI, 2025] self-evolves with no cold-start; R1 adds a small supervised seed. Off-policy guidance that mixes expert trajectories into on-policy RL [Yan et al., 2025] is the lineage of our demonstration-mixing. Our contribution is to *locate the boundary* between the two regimes empirically: reward-only RLVP suffices on discoverable workflows, and a demonstration becomes necessary precisely when a required behavior is never sampled—a wall we exhibit with a silent-precondition task. Customer-service benchmarks with explicit policy documents and reliability metrics [Yao

et al., 2024] motivate both our real-benchmark study and the outcome-instrumental rules our boundary analysis identifies as future work.

10 Conclusion

Outcome-only reinforcement learning is silent exactly when long-horizon agents most need to be shaped, because group-relative advantages vanish on the all-fail groups that dominate early training. Reinforcement Learning from Verifiable Penalties fills that silence with a dense, machine-checkable process signal that produces gradient from failed episodes—and the payoff is measured in the outcome itself: a several-fold reduction in samples to a target success rate, with near-zero variance, against both vanilla GRPO and DAPO's discard-and-resample. We isolated the mechanism in a paired experiment, showed the credit must be attached to the responsible tokens, and showed the signal can be auto-derived from tool metadata with no human-written rules.

We were equally concerned to map the edges. Self-evolution has a discovery ceiling: when a required precondition is never sampled, no reward-only method escapes, and a single demonstration is what breaks the wall—a clean empirical line between what reward can grow and what imitation must seed. And process rewards accelerate the *outcome* in proportion to how much of what the reward requires the rules can verifiably cover—a coverage gradient with an irreducible ceiling. Pointed at a real benchmark, rules orthogonal to the reward drive a degenerate compliance-only collapse; policy-derived rules, both procedural and verifiable-semantic, remove that harm; but neither beats outcome-only, because verifiable rules express *general policy*—procedure and validity—while the reward's residual difficulty is *task-specific intent*, which is the reward itself and learnable only from the outcome signal. The method is therefore not a universal accelerant but a precise one: it shines where the verifiable workflow is essentially the whole task, neutralizes harm where it is not, and the line it draws between policy that rules can encode and intent that only outcome can teach is, we believe, the more durable contribution.

References

Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.

DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

Lang Feng et al. Group-in-group policy optimization for llm agent training. *arXiv preprint arXiv:2505.10978*, 2025.

Anisha Gunjal et al. Rubrics as rewards: Reinforcement learning beyond verifiable domains. *arXiv preprint arXiv:2507.17746*, 2025.

Harrison Lee, Samrat Phatale, Hassan Mansoor, Kellie Lu, Thomas Mesnard, Colton Bishop, Victor Carbune, and Abhinav Rastogi. RLAIF: Scaling reinforcement learning from human feedback with ai feedback. *arXiv preprint arXiv:2309.00267*, 2023.

Cheng Qian et al. Toolrl: Reward is all tool learning needs. *arXiv preprint arXiv:2504.13958*, 2025.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Jianhao Yan et al. Learning to reason under off-policy guidance. *arXiv preprint arXiv:2504.14945*, 2025.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.

Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.

Yuanqing Yu et al. Steptool: Enhancing multi-step tool usage in llms through step-grained reinforcement learning. *arXiv preprint arXiv:2410.07745*, 2024.

Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-rewarding language models. *arXiv preprint arXiv:2401.10020*, 2024.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *arXiv preprint arXiv:2306.05685*, 2023.